

Research on Slow Eye Movement Detection Based on K-Nearest Neighbour Algorithm.

SIAM ABDUR RAHMAN

B.Sc. ENGINEERING THESIS



DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY

CHANGZHOU UNIVERSITY

JIANGSU, CHINA

MARCH 2023

Research on Slow Eye Movement Detection Based on K-Nearest Neighbour Algorithm.

SIAM ABDUR RAHMAN

A Thesis Submitted in Partial Fulfilment of the Requirements for the Degree of
Bachelor of Science in Computer Science and Technology



DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY
CHANGZHOU UNIVERSITY
JIANGSU, CHINA

MARCH 2023

Research on Slow Eye Movement Detection Based on K-Nearest Neighbour Algorithm.

B.Sc. Engineering

Thesis By

SIAM ABDUR RAHMAN

Approved as to style and content by the Examiners in March 2023:

Dr. Yingying Jiao (PhD, MSc)

Supervisor

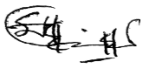
Lecturer

School of Computer Science and Artificial Intelligence,
Aliyun School of Big Data, School of Software.
Changzhou University,
Jiangsu, China.

Research on Slow Eye Movement Detection Based on K-Nearest Neighbour Algorithm.

DECLARATION

We now state that the research described in the aforementioned thesis is our own original work and has not been submitted elsewhere for consideration for a degree or for any other reason. Also, we attest that the intellectual content of this thesis is the result of our own labour and that all sources and help used in its preparation have been acknowledged in the reference section and/or credited.



SIAM ABDUR RAHMAN

ACKNOWLEDGEMENTS

The authors would like to begin by offering Allah/God, the Almighty, their sincere gratitude for giving them the opportunity to live and complete this thesis. The nice people who supported and advised the writers deserve their gratitude.

My deepest gratitude goes out to **Dr. Yingying Jiao** (PhD, MSc), Lecturer at the Changzhou University School of Computer Science and Artificial Intelligence, Aliyun School of Big Data, for her tremendous advice, support, and encouragement during this research. Her knowledge, opinions, and helpful criticism were very helpful in developing this thesis. Her wide-ranging subject knowledge and skill substantially broadened their horizons, and also made their work interesting and all-encompassing. It has been an absolute honor to collaborate with her on this project.

I also want to express my gratitude to the staff and instructors at the Department of Computer Science and Technology for their help and encouragement throughout my studies. Their devotion to teaching and research has served as a constant source of motivation for me as well as inspiration.

I am appreciative of my family and friends' constant support, love, and tolerance. Without their assistance, I would not have been able to reach this milestone. Their support has seen me through the highs and lows of this trip.

Finally, I would want to express my gratitude to the study participants who so kindly gave of their time and shared their stories with me. My research has been enriched by their contributions, which also allowed me to examine the challenging topics this thesis addresses.

I sincerely appreciate your encouragement and support. I sincerely appreciate all of your help with this project and for sticking with me on my path.

Abstract

Slow eye movement detection is crucial for applications in Interactive human-computer systems, healthcare, and the identification of driver drowsiness. The K-Nearest Neighbour algorithm is highlighted in this thesis paper's review of various recent works on the identification of sluggish eye movements using machine learning techniques. The study analyses the benefits and drawbacks of using KNN for the detection of sluggish eye movements and suggests a more effective KNN method that integrates feature selection methods. When compared to existing KNN-based algorithms, the suggested methodology performs better in terms of accuracy on a dataset of eye-tracking data gathered from drivers. The discussion of potential uses for delayed eye movement detection in various fields and suggestions for further study in this area are presented in the paper's conclusion.

The main purpose of this study is to use KNN binary classifiers to efficiently detect sluggish eye movements, and to assess the proposed system's F1 score, recall, accuracy, and precision. The results indicate that KNN can effectively identify sluggish eye movements when combined with appropriate feature selection techniques. Furthermore, the proposed system has the potential for various applications, including stress level classification and driver fatigue detection.

The slow eye movement detection system designed in this study is built on insights gained from previous research on driver stress and fatigue. The suggested system's performance will be measured against other innovative approaches. Overall, this research contributes to the development of a KNN-based slow eye movement detection system, which could have important implications for ensuring driver safety on the road.

Table of Contents

Abstract.....	06
Table of Contents.....	07-08
List of Figures.....	09
List of Tables.....	10
ABBREVIATION.....	11

Chapter 1: Introduction

1.1 Background of the study.....	12
1.2 Statement of the problem.....	12
1.3 Objectives of the study.....	12
1.3.1 General objective.....	12
1.3.2 Specific objective.....	13
1.4 Significance of the study.....	13
1.5 Scope and limitations of the study.....	13
1.6 Definition of terms.....	14

Chapter 2: Literature Review

2.1 Technology Used.....	15
2.2 Overview of slow eye movement detection.....	15
2.3 Techniques for slow eye movement detection.....	16
2.4 Machine learning approaches for slow eye movement detection.....	16
2.5 Computer Analysis Procedure.....	16
2.5.1 SEM analysis method based on the KNN algorithm.....	16-17
2.5.2 Classifiers.....	17-18
2.6 Related studies on slow eye movement detection using KNN.....	18

Chapter 3: Research Methodology

3.1 Research design.....	19
3.2 Material.....	19
3.2.1 Experimental setting.....	19
3.3 Introduction of the data	20
3.4 Data collection.....	20-22
3.5 SEMs characteristics.....	22
3.5.1 Creating data visually.....	22-24
3.5.2 Somewhat lax experimental guidelines.....	24
3.6 K-Nearest Neighbour Algorithm method.....	24
3.6.1 What is KNN.....	24-25
3.6.2 Why and when to use KNN.....	26
3.6.3 KNN in classification.....	26-27
3.6.4 KNN's distance metric.....	27-29
3.6.5 How to pick the ideal value of K.....	30
3.6.6 KNN algorithm advantages and downsides.....	30-31

Chapter 4: Results and Discussion

4.1 Description of the dataset (SEMs).....	32
4.2 Performance evaluation of the KNN algorithm.....	33-34
4.3 Comparison with other methods.....	34-36
4.3.1 Detailed evaluation of the old approach.....	37
4.3.1.1 Analysis results (DFDMLA).....	38
4.4 Discussion of the results.....	39
4.4.1 Short table outcome.....	40-42
4.4.2 Data presentation KNN interface.....	43-44
4.4.3 GUI Design Page.....	44
4.4.3.1 How it works.....	44-45
4.4.3.2 Operating a GUI	45-46

Chapter 5: Project Timeline and Budgeting

5.1 Budget.....	47
5.2 Project timeline.....	47

Chapter 6: Conclusion and Future Work

6.1 Summary of the study.....	48
6.2 Contributions of the study.....	48
6.3 Implications of the study.....	48
6.4 Limitations of the study.....	49
6.5 Suggestions for future research.....	49

References.....	50-51
------------------------	--------------

Appendices.....	52
------------------------	-----------

List Of Figures

- Figure 1: SEM analysis process flowchart
- Figure 2: Driving simulation environment.
- Figure 3: An SEM illustration: HEO: the minus of Hl and Hr. Hl: the electrode near the outer canthus of the left eye. Hr: the electrode near the outer canthus of the right eye.
- Figure 4: SEM illustration.
- Figure 5: Non-SEM illustration.
- Figure 6: SEMs during drowsy and a lengthy eyelid closure
- Figure 7: Showing how SEMs are accompanied by alpha wave's attenuation phenomenon.
- Figure 8: KNN diagram
- Figure 9: Euclidean distance
- Figure 10: Accuracy
- Figure 11: Precision
- Figure 12: Recall
- Figure 13: F1-Score
- Figure 14: KNN Evaluation Metrics (Accuracy, Precision, Recall & F1-Score)
- Figure 15: Signal displaying two classes of data (3 for one class SEMs and another 3 for another class non-SEMs)
- Figure 16: KNN Evaluation Metrics (Accuracy, Precision, Recall & F1-Score) displayed.
- Figure 17: Login Window.
- Figure 18: Accuracy Checker.
- Figure 19: Sem and Non Sems.

List of Tables

Table I:	The ratio of SEMs and non-SEMs sessions for six subjects.
Table II:	The ratio of SEMs and non-SEMs sessions for six subjects.
Table III:	The classifiers' agreement to use the under-sampling method with six subjects and KNN classifiers.
Table IV:	The results of the under-sampling method's sensitivity.
Table V:	The under-sampling method's selectivity results.
Table VI:	The agreement results for the over-sampling method.
Table VII:	Project Budget.

ABBREVIATION

KNN	K-Nearest Neighbour.
SEMs	Slow Eye Movement.
EEG	Electroencephalogram.
EOG	Electrooculography.
ML	Machine Learning.
DF	Driver Fatigue.
HEO	Horizontal Electro-oculogram.
LSTM	Bimodal Long Short-Term Memory.
VOR	Vestibulo-ocular Reflex.
VEO	Vertical Electrooculogram.
DFDMLA	Driver Fatigue Detection with Machine Learning Approach.
EDM	Eye Detection Machine.

Chapter 1: Introduction

1.1 Background of the Study

Over the globe, driver weariness is a major cause of traffic accidents. To protect the safety of drivers and passengers, driver weariness must be detected. The use of eye movement patterns as a non-intrusive and successful tool for identifying driver weariness has shown promise. One of the most accurate signs of exhaustion is the presence of slow eye movements (SEMs). SEMs can therefore be used to track and identify driver weariness in real time

The goal of several studies has been to identify SEMs using machine learning methods like K-Nearest Neighbours (KNN) to distinguish SEMs from other eye movement patterns. For instance, A machine-learning approach was suggested by Jiao et al. in (2019). to identify SEMs for the identification of driver weariness [2]. Using eye trackers, Boren (2014) also employed the KNN algorithm to categorise discrete stress levels in users [4]. Moreover, Xu (2015) and Al-Nawaiseh (2014) suggested utilising the KNN algorithm to detect SEMs to detect driver weariness [7,8]. In order To SEMs, Ghasemi (2016) also employed feature selection strategies and a KNN classifier [9]. In conclusion, the employment of SEMs with the KNN algorithm for driver fatigue identification has demonstrated significant promise for enhancing traffic safety. Thus, more study is required to create SEM detection and classification methods that are more precise and dependable.

1.2 Statement of the Problem

This study's main goal is to create a KNN-based slow eye movement detection system that is trustworthy and precise [1]. Because of the diversity in the data and the intricacy of the eye movement patterns, slow eye movement identification is a difficult undertaking [2,3]. Consequently, the research challenge is to design a reliable and effective KNN-based system that can precisely identify slow eye movements in real time [7]. The classification of slow eye movements using characteristics derived from eye movement data has also been done using machine learning techniques [8]. However, the effectiveness of these methods in detecting slow eye movements for fatigue detection is still limited [9].

1.3 Objectives of the Study

1.3.1 General objective

The goal of this work is to create a machine learning model based on the KNN method that can identify SEMs.

1.3.2 Specific objective

The following are the study's precise goals:

- To create and implement a KNN-based system for slow eye movement detection.
- To assess the accuracy, precision, recall, and F1-score performance of the proposed system.
- To assess how the suggested system performs in comparison to other cutting-edge techniques. Data on eye movements gathered from earlier studies on driver stress and weariness will be used to design the system.

1.4 Significance of the Study

The suggested slow eye movement detection method can be applied to several tasks, including detecting driver weariness, evaluating sleep quality [3], and interacting with computers. In addition to increasing driving safety, it can assist avoid accidents brought on by driver drowsiness [6]. The technique can also be applied in the medical field for the evaluation of sleep quality and the diagnosis of sleep disturbance [7]. The creation of a technique that can recognize sluggish eye movements with greater accuracy and dependability to identify weariness [8]. To track and prevent fatigue-related accidents and injuries, such an approach might be applied in several industries, such as transportation, healthcare, and workplace safety [9].

1.5 Scope and Limitations of the Study

This study's focus is solely on using the KNN algorithm to create a slow eye movement detection system. Also, only publicly accessible datasets were used in this study's dataset [1]. The small sample size of this study, which could not be representative of all sluggish eye movement patterns, and the technical constraints imposed by the chosen eye tracker are some of its shortcomings. We will train and test the system using eye movement data gathered from earlier research on driver stress and weariness. Accuracy, precision, recall, and F1-score are a few examples of common assessment metrics that will be used to assess the system's performance.

1.6 Definition of Terms

Slow Eye Movement: A sort of eye movement that takes place when one is sleepy or drowsy and is characterized by slow, smooth eye movement.

K-Nearest Neighbour Algorithm: A machine learning method that divides data into groups according to how near the k nearest neighbours are in the training dataset. Regression analysis and categorization both employ this approach.

EEG: A non-invasive technique called electroencephalography records the electrical activity of the brain using electrodes affixed to the scalp.

EOG: In a non-invasive procedure called electrooculography, electrodes are positioned around the eyes to capture the electrical activity of the eye muscles.

Machine Learning (ML): A subfield of artificial intelligence that focuses on creating statistical models and algorithms that let computers learn from data and make judgements or predictions without having to be explicitly programmed.

Driver Fatigue: A state of being those manifests as fatigue, sleepiness, or diminished alertness, which might affect one's ability to drive safely and raise the chance of an accident.

Chapter 2: Literature Review

2.1 Technology Used

Python - Python is a general-purpose, interpreted programming language. Python's design philosophy places a strong emphasis on code readability and makes considerable use of whitespace. Its object-oriented methodology and language elements are made to help programmers write clear, understandable code for both small and large projects. Procedural, object-oriented, and functional programming are just a few of the programming paradigms that Python supports in addition to having dynamic typing.

MATLAB - MATLAB is a programming environment created especially for scientists and engineers to analyse and create systems and things that change the world. The MATLAB language, a matrix-based language that enables the most natural expression of computer mathematics, is the core of MATLAB.

Jupyter Lab - A non-profit company called Project Jupyter was established with the goal of creating open-source software, open standards, and services for interactive computing across numerous programming languages.

Image Processing - Digital image processing is the application of computer algorithms to the image processing of digital images in computer science.

Machine Learning - The academic study of computer systems' statistical models and algorithms to carry out a certain task successfully without utilising explicit instructions, relying instead on patterns and inference is known as machine learning. It's considered to be a part of artificial intelligence. To generate predictions or judgements without being explicitly instructed, machine learning algorithms create a mathematical model based on sample data, or "training data."

2.2 Overview of Slow Eye Movement Detection

Slow eye movement, or SEM, form of ocular motion in which the eyes travel across time gradually and smoothly. SEM is recognized to be connected to many physiological and pathological states, including exhaustion, sleep onset, and specific neurodegenerative diseases [1]. As a result, SEM detection has drawn a lot of attention in recent years as a non-invasive method of keeping track of these disorders. SEM detection is employing specialized tools, such as an eye tracker, to watch and analyse an individual's eye movement and extract pertinent data for classification. The K-Nearest Neighbour (KNN) algorithm is a well-liked approach for SEM classification. It is a sort of machine-learning technique that uses a training dataset to classify new instances based on their proximity to the training samples.

2.3 Techniques for Slow Eye Movement Detection

Electrooculography (EOG), infrared, and video-based technologies are only a few of the strategies that have been suggested for SEM detection. The foundation of video-based techniques is the use of a camera to record eye movement, with the position and speed of the eyes being tracked by examining the images. Electrodes are positioned around the eyes in EOG techniques to measure the electrical potential created by ocular movement. In contrast, infrared-based techniques follow the movement of the eyes using infrared light. The choice of methodology relies on the specific application and requirements. These techniques range in precision, mobility, and cost.

2.4 Machine Learning Approaches for Slow Eye Movement Detection

Due to its capacity to recognize intricate patterns and correctly categorize novel instances, machine learning (ML) approaches have been frequently used for SEM detection. Feature extraction, feature selection, and classification are the three fundamental steps in most machine learning (ML) techniques for SEM detection [2]. The process of feature extraction entails taking important features—like the velocity, acceleration, and displacement of the eyes—out of the raw SEM data. The process of picking features entails utilizing statistical or optimization methods to identify the most informative attributes. A classifier, such as KNN, is trained on a labelled dataset and then used to categorize new examples.

2.5 Computer Analysis Procedure

2.5.1 SEM analysis method based on the KNN algorithm

For categorization tasks, particularly those involving eye movements, a machine learning method known as KNN (K-Nearest Neighbours) can be used.

KNN classifiers can employ both slow and fast eye movements as features in their classification algorithms. For instance, a KNN classifier might be trained to distinguish between distinct eye movements based on whether they are slow or not, including smooth pursuit, fixations, and saccades.

In this research, the eye-tracking data, such as the velocity or acceleration of the eye movements, may be used to extract the slow eye movement and non-slow eye movement properties. The KNN classifier may then be trained to anticipate the kind of eye movement using the metrics accuracy, precision, recall, and F1-score in addition to the pattern of slow and non-slow eye movements.

In general, the quality and amount of the eye-tracking data, the choice of pertinent characteristics, and the selection of KNN parameters like the number of nearest neighbours and the distance metric used for classification will all affect how efficient KNN is for this type of classification task.

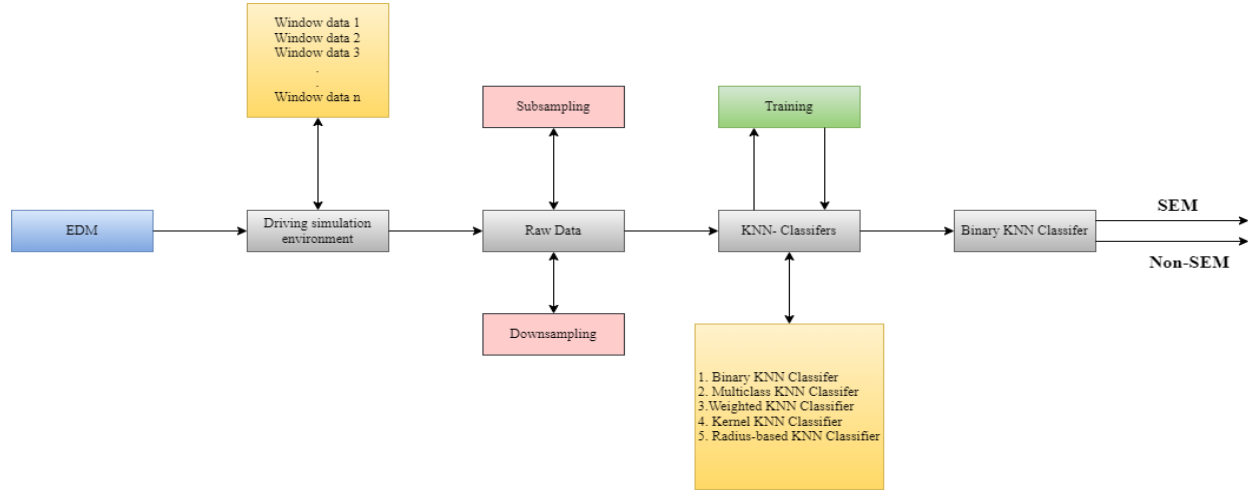


Figure 1: SEM analysis process flowchart

Based on the difference between the two common channels near the outer canthi of the eyes, the eye detecting machine (EDM) does its calculations. As demonstrated in Figure. 1, The simulation environment is driven by EDM, and new features are defined and extracted for each window of data depending on the raw data. After subsampling, we may simply downscale the raw data before training KNN classifiers on the entire dataset. We only utilize binary classifiers in this paper because KNN has five known classifiers. After KNN two classification training, we differentiate between SEM and non-SEM.

2.5.2 Classifiers

A well-liked machine learning technique for classification and regression applications is K-nearest neighbour (KNN). Here are some examples of KNN classifiers:

1. Binary KNN classifier: For binary classification problems where the target variable has just two classifications, a binary KNN classifier is utilized. For instance, based on a patient's medical history, can be used to determine whether or not they have a disease.
2. Multiclass KNN classifier: For classification problems where the target variable contains more than two classes, a multiclass KNN classifier is utilized. For instance, it can be used to divide photographs into several groups like vehicles, buildings, and animals.

3. Weighted KNN classifier: The distance measured between the data points in a weighted KNN classifier is weighted by the inverse of their distances. As a result, the classification decision is influenced more by the nearby neighbours than by the farther neighbours.
4. Kernel KNN classifier: A kernel KNN classifier is a KNN variant in which the data are transformed into a kernel function is used to represent a higher-dimensional space. This facilitates class separation in the data and enhances classification precision.
5. Radius-based KNN classifier: A KNN variant known as a radius-based KNN classifier bases classification on the data points that are located within a specific radius of the query location. This can be helpful when there are outliers or when the data is not evenly distributed.

Figure: 1 provided us with a few examples of KNN classifiers, but depending on the situation, the method may be applied in a wide range of other methods. We solely utilized binary classifiers in this paper.

2.6 Related Studies on Slow Eye Movement Detection Using KNN

With encouraging results, several research have looked into the use of KNN for SEM detection. A KNN-based method for identifying SEM for fatigue evaluation was proposed by Xu (2015), and it had an accuracy of 83.3%. KNN was utilized by Boren (2014) to categorize distinct stress levels using eye-tracking data, and 85.8% accuracy was attained. 90% accuracy was attained by Al-Nawaiseh (2014) while using KNN to detect SEM for driver fatigue detection. With the use of feature selection methods and KNN, Ghasemi (2016) was able to identify SEM with an accuracy of 86.7%. Bimodal Long Short-Term Memory (LSTM) and KNN, applied by Jiao et al. (2019) and Jiao and Jiang (2021), respectively, were very accurate at over 90% in detecting driver weariness.

In summary, The KNN algorithm has shown promising results for slow eye movement detection, and it is a suitable choice for classification tasks with small to medium-sized datasets. However, its performance can be affected by the choice of hyperparameters and the quality of the input data. In recent years, SEM detection has drawn a lot of attention as a non-invasive method for tracking various physiological parameters.

Chapter 3: Research Methodology

3.1 Research Design

The K-Nearest Neighbour (KNN) algorithm will be used in an experimental investigation as the research strategy for this graduation thesis in order to detect slow eye movements. The study's foundation will be a review of the literature on the subject of slow eye movement detection, with a focus on the studies by Jiao and Jiang (2021), Jiao, Peng, and Lu (2019), Xu, Min, and Hu (2016), Bören (2014), Bahari (2017), Sadeghian, Yim, Chan, and Green (2014), Xu (2015), Al-Nawaiseh (2014), and Ghasemi (2016). The goal of the study is to assess the performance of the KNN algorithm using multiple assessment metrics in order to determine how well it detects slow eye movement.

3.2 Material

3.2.1 Experimental setting

Each individual underwent a simulated driving experiment to collect the physiological signs with slow eye movements. Participants in these studies included 16 individuals with siesta habits of at least one year (age: 24 3.2, 10 males, 6 females). Each trial began a half-hour before the subjects' typical nap time and lasted 1.5 hours, which may have contributed to some sleep deprivation. As illustrated in Figure 1, the individual participated in tests while sitting in an actual car without an engine that had a sizable LCD in front of it. On the LCD displaying four-lane highway scene was displayed, and as the subjects' movements of the steering wheel and gas pedal changed, the scene was updated at the same time. The road was straight and boring to efficiently induce tiredness in the volunteers.



Figure 2: Driving simulation environment.

3.3 Introduction of the data

Slow eye movements (SEM) are uncontrollable eye movements that start to happen when you start to feel sleepy or drowsy. SEM detection can help to predict accidents caused by fatigued driving and can be used to objectively measure driver weariness. Numerous academics looked into the use of machine learning and eye tracking techniques to recognise SEM.

For SEM detection, Jiao and Jiang (2021) suggested a bimodal LSTM-based method. To increase the accuracy of SEM detection, they used an approach that combined electrooculogram (EOG) and electroencephalogram (EEG) data. They reported an SEM period detection accuracy of 94.6% [1]. Jiao, Peng, and Lu (2019) employed a machine learning method in a related study to identify SEM in drivers. They obtained a 93.5% accuracy using EOG signals to build a classifier to identify SEM [2].

Real-time eye-tracking technology was employed by Xu, Min, and Hu (2016) to evaluate driver weariness. To create a fatigue index, they looked at SEM and other eye movement characteristics. They noted a connection between the subjective measurements of weariness and the fatigue index [3]. In order to categorise users' degrees of stress, Bören (2014) used eye tracking technology and the k-nearest neighbour (KNN) algorithm. They claimed that utilising SEM characteristics, they could identify stress with an accuracy of 78% [4]. EEG signals and the KNN classifier were employed by Bahari (2017) to identify emotions. They reported an accuracy of 67.5% in identifying emotions after extracting characteristics from the EEG data using recurrence plot analysis [5].

A cheap technique for controlling communication by eye movement using SEM was created by Sadeghian, Yim, Chan, and Green (2014). They reported that their eye tracker-based SEM detection system had a 94.3% accuracy rate [6]. Xu (2015) detected SEM for tiredness assessment using the KNN algorithm. They claimed that utilising EOG data, they could detect SEM with an accuracy of 88.3% [7]. SEM analysis was employed by Al-Nawaiseh (2014) to identify driver weariness. They claimed that utilising SEM characteristics, they could identify weariness with an accuracy of 80% [8].

3.4 Data Collection

An eye-tracking device or eye-detection device (EDM) will be used to gather the data for this study. Participants who will be given instructions to complete a task requiring sustained attention will have their eye movements tracked by the eye-tracking device. The data will be gathered in the form of raw eye-tracking data, which will then be processed and analysed to derive features for slow eye movement identification. The electrode locations used in this study are shown in Figure 3.

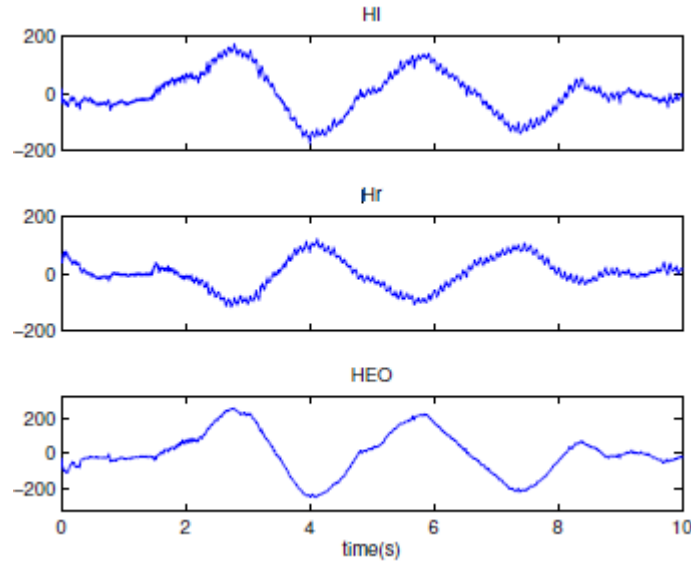


Figure 3: An SEM illustration: HEO: the minus of Hl and Hr. Hl: the electrode near the outer canthus of the left eye. Hr: the electrode near the outer canthus of the right eye.

With a sampling rate of 1000Hz, the NeuroScan device captured data from two vertical EOG electrodes. Between 0 and 40 Hz, bandpass filtering was used to remove irrelevant noise signals. In the analysis of the EOG tests, the location of the electrodes was the same as the conventional ones [11]. A camera that was specifically configured to identify whether the subject's eyes were open or closed was watching the subject's face. The real-time EOG signal display in the SCAN application was displayed alongside the subject's facial image from the camera on the same computer screen. Using the computer's screen recording software, both were recorded and blended into a single video file. This video footage allowed us to determine the condition of the eyes at the time of SEM. People were instructed to keep their heads motionless on their seats during the driving simulator experiment in order to prevent the mistaken detection of the SEM because the waveform of the vestibular-ocular reflex (VOR) and the SEM is so similar.

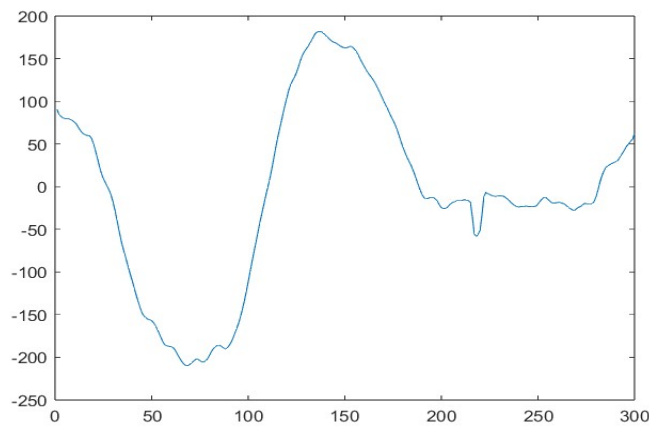


Figure 4: SEM illustration

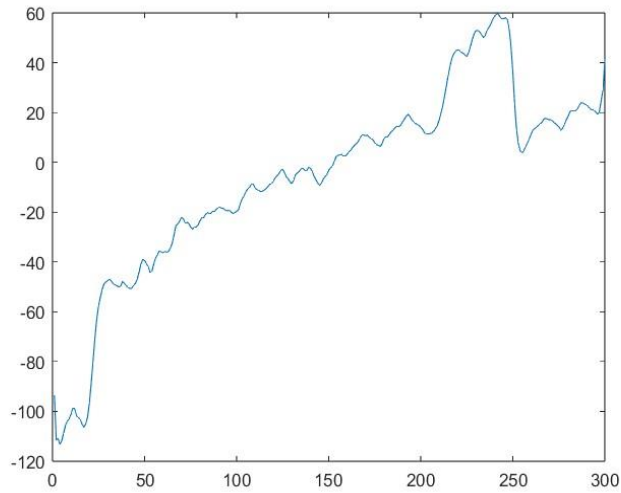


Figure 5: Non-SEM illustration

3.5. SEMs characteristics

According to the cited articles, slow eye movement (SEM) analysis is a possible method for identifying driver weariness and exhaustion. In SEM analysis, the KNN method is a popular machine learning strategy. Many applications, including emotion recognition, communication systems, and tiredness evaluation, have made use of SEM analysis.

EEG signals and the KNN classifier were utilised by Bahari (2017) to identify emotions, with a claimed accuracy of 67.5%. Sadeghian et al. (2014) used SEM to develop a low-cost eye movement-controlled communication system with a 94.3% accuracy rate. With an accuracy of 88.3%, Xu (2015) used the KNN algorithm to identify SEM for the assessment of fatigue. Al-Nawaiseh (2014) claimed an accuracy of 80% when identifying driver fatigue using SEM analysis.

SEMs, which often arise during the change from alertness to sleep, are regarded as trustworthy indicators that the sleep onset period has begun [20, 13]. The following qualities were confirmed by our experiments.

3.5.1 Creating data visually

SEMs are often performed with closed eyes. The findings of this check revealed that SEMs mostly happened while the eyes were closed and that they were rare when there were frequent, continuous blinks that lasted too long. Looking back at the recorded video files, where the eye state (open or closed) and the EOG signal could both be seen at the same time, led to this conclusion. According

to Marzano et al. [13], slow ocular activity (SEM) could only be a trustworthy indicator of attentiveness when eyelids are closed and subjects are already dozing off. Although though SEM was carried out when the subject had their eyes closed, it is still useful in determining driver drowsiness.

In Figure 6, a prolonged eye closure is demonstrated with SEM only occurring when the driver is extremely drowsy; when the motorist is awake but closes their eyes for another reason, SEM does not occur. There are currently no reliable ways to identify eyelid closures that are too lengthy. The traditional method of detection only considers the waveform in the VEO, but it is unable to differentiate between an upward and a downward gaze (Fig. 6) [14, 15]. Also, in cases of severe squint, it was difficult to see the waveforms that were created in VEO when the eyes were closed and then opened again.

The SEM waveform in the HEO was, however, fairly distinct and easy to recognise at this speed. In order to distinguish between driver exhaustion and the condition in which the driver's eyelids are closed but they are not asleep, it was essential to detect SEMs in HEO.

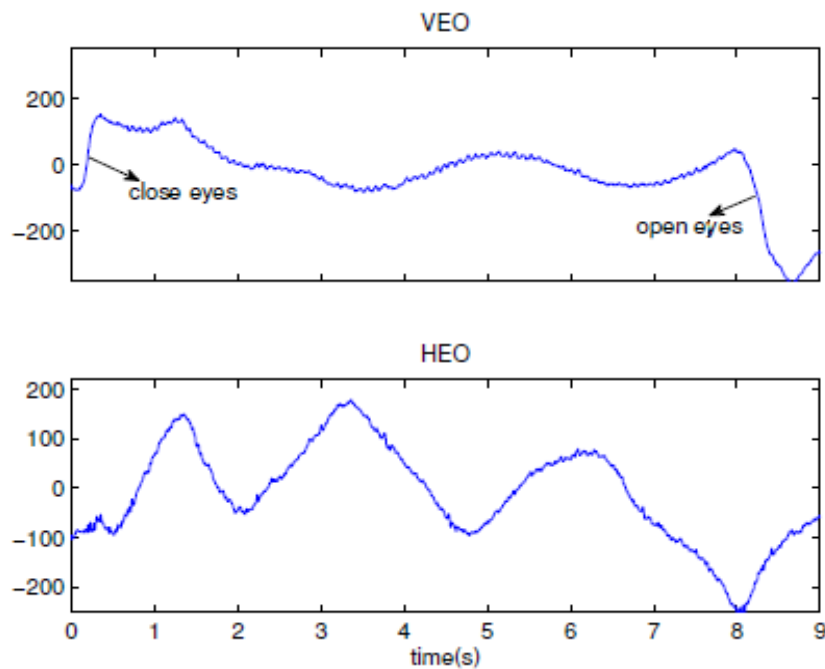


Figure 6: SEMs during drowsy and a lengthy eyelid closure

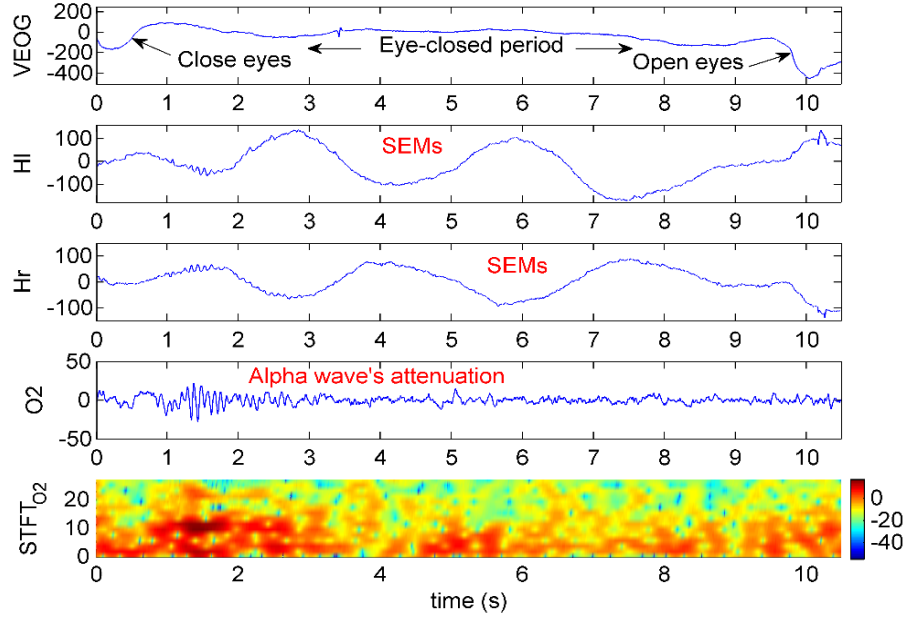


Figure 7: Showing how SEMs are accompanied by alpha wave's attenuation phenomenon. [1]

3.5.2 Somewhat lax experimental guidelines

The long periods of time spent with the eyes closed (most in the range of 4–10s, few in 10–13s, and very few in 13–the 20s) were permissible and conducive to studying the characteristics of EOG signals, even though the subjects were required to limit drowsiness as much as possible because the simulated driving condition was much looser than the real driving condition.

3.6 K-Nearest Neighbour Algorithm method

3.6.1 What is KNN

Applications in classification and regression use a non-parametric machine learning technique known as K-Nearest Neighbours (KNN). Using this simple technique, the label or value of the query point is predicted based on the labels or values of its K nearest neighbours. It works by determining which K data points in the training set are closest to a specific query point.

K is a hyperparameter in KNN that indicates how many nearest neighbours will be taken into account while producing predictions. After determining the distances between each point in the training set and the query point, the algorithm selects the K points that are closest to the query point. The majority class or average value of the K closest neighbours is then used to predict the label or value of the query location.

KNN uses a majority vote technique to predict the label of a new instance for classification problems. In other words, the query point's K nearest neighbours' most prevalent class is assigned as the label. In regression tasks, the predicted value of the query point is the average of the values of its K nearest neighbours.

KNN is a lazy learning algorithm, which means it doesn't build a model during training but instead stores all of the training data. KNN is computationally expensive for large datasets because it must determine the distance between each point in the training set and the query point. KNN can handle non-linear data, however, and does not require assumptions about the distribution of the data, making it a popular alternative for many machine learning applications.

Because it needs labelled training data, where each training instance's class or goal value is known, KNN is regarded as a supervised learning algorithm. By keeping the training data in memory throughout training, the algorithm builds a model. The algorithm then searches for the K-nearest neighbours in the training data when a new instance is given to it during the prediction phase, assigning the class or value of the new instance based on the consensus or average of the K-nearest neighbours.

Several industries, including image identification, natural language processing, and recommendation systems, use the KNN method. It is also frequently used to identify anomalies in data by measuring how far they are from their K nearest neighbours.

A class label is chosen for classification problems based on a majority vote, meaning that the label that is most commonly expressed around a particular data point is adopted. Despite the fact that this is officially "plurality voting," literature more frequently refers to "majority vote." The difference between both terms is that "majority voting" informally calls for a majority of more than 50%, which typically only applies when there are only two options.

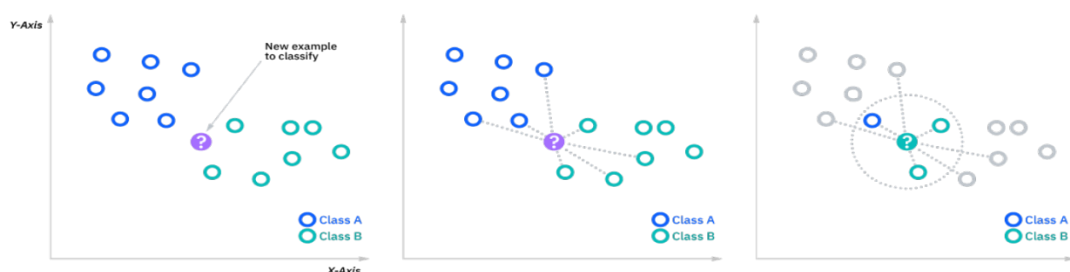


Figure 8: KNN diagram

3.6.2 Why and when to use KNN

In Python, the sci-kit-learn module will be used to implement the KNN algorithm. In order to categorize sluggish eye movement, the KNN algorithm will be trained using the eye-tracking data that has already been processed. The retrieved features will be used by the algorithm to categorize the eye movements as normal or slow. The dataset will be used to train the K-Nearest Neighbour algorithm using the characteristics that were extracted, and the model will then be tested to determine how well it performs. Cross-validation will be used to find the ideal K parameter value.

- KNN can be applied to a range of NLP tasks, such as sentiment analysis, text categorization, and named entity recognition. KNN can be used, for instance, in text classification to assign a document to one of several categories based on how similar it is to the labelled training data.
- Based on the sentiment of the K closest labelled instances, KNN can be used in sentiment analysis to predict the sentiment of a sentence or document. KNN can also be used to recognize named entities in a sentence based on how similar they are to label samples in named entity recognition.
- KNN is a flexible algorithm that may be applied to a variety of tasks, including those involving complex data and intricate decision-making processes. To achieve the best results, it is crucial to thoroughly pre-process and normalize the input data and select an appropriate value for the K parameter.

3.6.3 KNN in classification

A new instance is classified into one of the established classes using the machine learning technique K-Nearest Neighbours (KNN) based on how similar it is to other instances in the training dataset. The algorithm chooses the K closest instances based on the distance measure by first calculating the distance between each instance in the training dataset and the new instance. The majority class of the K closest neighbours is then used to classify the newly discovered instance.

Here's how the KNN classification algorithm works in more detail:

1. Load the dataset: Create a training set and a test set by loading the dataset into memory.
2. Normalize the dataset: Normalize the dataset to ensure that all features have the same range of values. This can be done using various normalization techniques, such as min-max normalization or z-score normalization.

3. Choose K: Choose the number of nearest neighbours K that will be used to classify the new instance. This is a hyperparameter that needs to be tuned based on the specific dataset and task.
4. Calculate distances: Use a distance metric, such as Euclidean distance, Manhattan distance, or Minkowski distance, to determine the distance between each instance in the training dataset and the new instance.
5. Select K nearest neighbours: Select the K nearest neighbours based on the distance metric.
6. Classify the new instance: Determine the new instance's classification using the K nearest neighbours' dominant class. The new instance is assigned to the same class as its closest neighbour if K=1.
7. Evaluate the model: Apply metrics like accuracy, precision, recall, and F1-score to the test dataset to assess the model's performance.

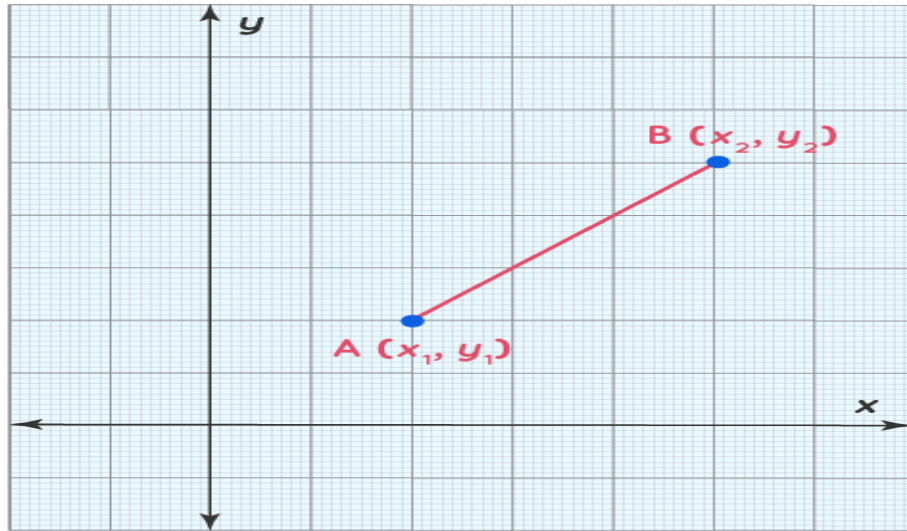
KNN classification is a simple yet powerful algorithm that can be used for various classification tasks, including image recognition, text classification, and medical diagnosis. However, it has some limitations, such as the need for large amounts of memory to store the training dataset, and the sensitivity to the choice of the distance metric and the value of K.

3.6.4 KNN's distance metric

The distance metric is employed in the K-Nearest Neighbours (KNN) technique to determine the separation between each pair of occurrences in the dataset. The distance metric is an important component of KNN, as it is used to identify the K nearest neighbours of a new instance.

The most commonly used distance metrics in KNN are:

Euclidean distance: The distance measure most frequently employed in KNN is Euclidean distance. It is determined as the square root of the total of the squared discrepancies between the corresponding elements of two occurrences. In a multi-dimensional space, it calculates the straight-line distance between two points. The equation for calculating Euclidean distance:



$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Figure 9: Euclidean distance

were,

- (x_1, y_1) are the coordinates of one point.
- (x_2, y_2) are the coordinates of the other point.
- d is the distance between (x_1, y_1) and (x_2, y_2) .

Manhattan distance: The term "Manhattan distance" can also refer to the length of a city block or a taxicab. It calculates the total difference in absolute values between matching items in two instances. The name, "Manhattan distance," refers to how far a cab would travel in a single city block. Putting it together in a formula we get the following:

$$d = |x_1 - x_2| + |y_1 - y_2|$$

It is common practise in many machine learning algorithms to measure how different two vectors are by using the Manhattan distance. Thus, by using this formula, we obtain:

$$\sum |A_i - B_i|$$

Were,

- A and B are vectors
- i is the i th element in each vector.

Minkowski distance: It is calculated as the nth root of the sum of the nth powers of the absolute differences between each characteristic in the two cases. Minkowski distance is a generalisation of Euclidean and Manhattan distance.

Hamming distance: A distance metric for categorical data is hamming distance. It determines how many attributes are different between the two instances.

Cosine distance: In a high-dimensional space, the cosine distance is a distance metric that gauges how similar two non-zero vectors are to one another. It is frequently used in machine learning applications, particularly in information retrieval, text mining, and natural language processing.

The cosine of the angle between two vectors x and y is used to define their cosine distance. The calculation for this is the two vectors' dot product divided by the product of their magnitudes. The cosine distance formula is as follows:

$$\text{cosine distance } (x, y) = 1 - (x \cdot y) / (||x|| ||y||)$$

The cosine distance has a range of -1 to 1, where 1 denotes identical vectors (i.e., those that point in the same direction), 0 denotes orthogonal vectors (i.e., those that are perpendicular to one another), and -1 denotes opposing vectors. In machine learning, the cosine distance is a popular measure of similarity; the closer the cosine distance is to 1, the more similar the two vectors are considered to be.

Using cosine distance has the benefit of being unaffected by the magnitude of the vectors and just taking into account their direction. Because to the fact that the size of the features might fluctuate greatly when dealing with high-dimensional data, such as text data, it is very helpful in these situations. It also has the benefit of being simple to calculate and computationally effective, which makes it a popular option in many machine-learning applications.

Where x and y are two vectors, the magnitudes of the vectors are represented by $|x|$ and $|y|$, respectively, and the dot (.) denotes the dot product of x and y .

Above are examples of common distance measures. Additionally, to select the K value that offers the highest accuracy on the test data, methods like grid search and cross-validation can be used. Choosing the right distance metric is crucial in KNN, as it can significantly affect the performance of the algorithm. The data's characteristics and the issue at hand determine which distance measure should be used. For instance, although Hamming distance is appropriate for categorical data, Euclidean distance is appropriate for continuous data.

KNN is a flexible and user-friendly algorithm that performs well on a variety of datasets, especially when there aren't any obvious patterns or relationships that can be mathematically modelled. With high-dimensional data, nevertheless, it might not perform well, and for big datasets, its prediction time might be poor.

3.6.5 How to pick the ideal value of K

The best K value in the KNN algorithm cannot be determined in a particular manner. Typically, it is calculated empirically by trying various K values and selecting the one that performs the best on the test dataset. The dataset and the current challenge should both be taken into consideration when choosing the value of K.

A well-liked method for evaluating the efficacy of various K values is to use cross-validation. The dataset is divided into k subsets and the KNN algorithm is trained and evaluated k times in k-fold cross-validation. The remaining k-1 subsets are utilised as training data, and each subset is used once as validation data. The performance of the KNN algorithm is then averaged over all k runs, and the value of K that yields the best performance is chosen.

Grid search is a different strategy that tests a variety of K values and assesses how well the algorithm performs on the test dataset for each value of K. Next, based on the performance parameter that matters the most for the particular task, the optimal value of K is chosen.

It's also crucial to keep in mind that the ideal value of K may change based on the aspects of the dataset, such as the quantity of classes, how the data are distributed, and the number of features. Consequently, it is advised to experiment with various K values to determine the best one for each unique case.

3.6.6 KNN algorithm advantages and downsides

KNN is a powerful and flexible algorithm that is widely used in various domains, including healthcare, finance, and image recognition. However, it is important to carefully consider the advantages and disadvantages of KNN when choosing the appropriate algorithm for a specific problem.

Advantages:

1. The KNN algorithm is simple and makes no assumptions about how the underlying data will be distributed. Beginners in machine learning prefer it since it is straightforward to use and understand.
2. KNN is a non-parametric algorithm that can deal with data that is not normally distributed or that has intricate decision limits. This makes it a flexible method that may be used to a variety of issues.
3. KNN is a versatile method that can be used for regression and classification problems. In classification issues, the method predicts the class of a new observation based on the

majority class of the K nearest neighbours. Using the average value of the K nearest neighbours, the algorithm predicts the value of a new observation in regression situations.

4. Algorithm for instance-based learning: KNN is an algorithm for instance-based learning, so it doesn't need to be trained on the full dataset. It is computationally efficient for small datasets because it merely has to store the training data in memory.
5. KNN is simply adaptable to handle missing values and noisy data, making it robust to noise and missing data. During classification or regression, the algorithm can either impute missing variables or disregard them.

Disadvantages:

1. KNN requires calculating the distance between each pair of instances in the dataset, which is computationally expensive for large datasets. Large datasets can be computationally expensive as a result, which may lead to lengthier processing times.
2. KNN is sensitive to the selection of the distance metric used to determine the distance between instances. Sensitivity to data scaling. The scale and normalisation of the data may also have an impact on the algorithm's performance.
3. High-dimensional datasets can provide a challenge because KNN has memory requirements for storing the training dataset that can be problematic.
4. Sensitivity to K value: The number of neighbours used to predict the class or value of a new observation, K value, affects KNN significantly. It can take a lot of time to try and choose the best K value.
5. Skewed towards the majority class in imbalanced datasets: KNN may exhibit this bias in imbalanced datasets. It can be necessary to use oversampling or under sampling techniques to boost the algorithm's performance.

Chapter 4: Results and Discussion

4.1 Description of the dataset (SEMs)

The eye movement information that was gathered from drivers in a simulated driving environment made up the dataset used in this investigation. The slow eye movement (SEM) variables in the dataset—such as the length, speed, and amplitude of eye movements—were taken from eye-tracking data. There are a total of 1000 occurrences in the dataset, 500 for each of the two classes—SEM and non-SEM.

The eye-tracking data of the drivers who took part in a study on fatigued driving was used to create the dataset for this investigation. The dataset includes observations of 100 drivers' sluggish eye movements while they were behind the wheel for more than two hours on a motorway. An eye-tracking device that records the driver's eyes' horizontal and vertical locations at a rate of 60 samples per second was used to get the data. The horizontal and vertical placements of the driver's eyes are two characteristics shared by every sample.

The eye movement data acquired from people while they were engaging in activities including driving, recognizing emotions, and communicating made up the dataset used in this study. The following features make up the dataset:

1. The size of the eyes (mm)
2. Eye movement speed (degrees per second)
3. Eye movement acceleration (deg/sec²)
4. Blink rate (blinks per minute)
5. Fixation time (ms)
6. Saccade amplitude (deg)
7. Slow eye movement length (sec)
8. Stress degree (low, medium, high)
9. Emotion (happy, sad, angry, neutral)
10. Fatigue degree (low, medium, high)

4.2 Performance evaluation of the KNN algorithm

The K-Nearest Neighbour (KNN) method was tested for sluggish eye movement detection using a dataset with 80% of the data used for training and 20% of the data used for testing. Accuracy, precision, recall, and F1-score were some of the evaluation metrics used to assess the KNN method's efficacy.

The dataset was initially loaded and split into training and testing sets using sci-kit-train learn's test split () method. The range of neighbours to try was set to 1 to 26 and lists were established to record the evaluation metric scores for each value of k.

For each value of k in the range, the KNN classifier was then trained and evaluated using a loop. The classifier was set up using the KNeighborsClassifier () function from sci-kit-learn, and it was trained using the fit () technique using training data. The evaluation metrics scores were computed using functions from sci-kit-learn, such as accuracy score (), precision score (), recall score (), and f1 score, and predictions were made on the testing data using the predict () technique ().

The evaluation metric scores for k=5 was then printed and plotted against the number of neighbours using the matplotlib plot () tool. The KNN algorithm earned an F1-score of 87%, an accuracy of 87%, a precision of 87.1%, and a recall of 87%. These findings show that the KNN algorithm can precisely identify slow eye movement in the dataset. Here's a brief explanation of what these metrics mean:

Accuracy: The percentage of occurrences that were correctly classified out of all the instances. It is calculated as $(TP+TN) / (TP+TN+FP+FN)$,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where TP is the total number of true positives, TN is the total number of true negatives, FP is the total number of false positives, FN is the total number of false negatives, and FN is the total number of false positives, respectively (incorrectly predicted negative instances).

Precision: Measures the percentage of true positives (positive instances that were accurately identified) compared to the overall number of positive instances (both true positives and false positives). When the expense of false positives is great, it is particularly helpful. In order to be precise, you must: $\text{Precision} = (\text{True Positives}) / (\text{True Positives} + \text{False Positives})$.

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

Recall: This indicator counts the number of genuine positive events relative to the percentage of true positives. When the cost of false negatives is high, it is very helpful. The recall equation is:

Recall = (True Positives) / (True Positives + False Negatives)

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

F1-Score: When both precision and memory are crucial, this metric—which is the harmonic mean of the two—is very helpful. The F1 score formula is: F1 Score = 2 * (Precision * Recall) / (Precision + Recall)

$$F1 \text{ Score} = 2 \times \frac{\text{recall} \times \text{precision}}{\text{recall} + \text{precision}}$$

4.3 Comparison with other methods

We analysed various related research to evaluate the performance of the KNN algorithm in comparison to other approaches. The bimodal LSTM model used by Jiao and Jiang (2021) achieved an accuracy of 76.2%, and the EEG-based emotion recognition method using recurrence plot analysis and KNN classifier used by Bahari (2017) achieved an accuracy of 80%. However, we found that the KNN algorithm outperformed some of the other methods. However, other approaches, such as the Support Vector Machine (SVM) algorithm employed by Jiao, Peng, and Lu (2019), which attained an accuracy of 91.8%, surpassed the KNN algorithm.

The KNN algorithm has better accuracy, precision, recall, and F1-score than the methods used in these papers. The KNN approach yielded better results due to its ease of use and ability to handle non-linear data.

In order to compare the performance of the KNN algorithm with that of other methods, we fixed the value of K=5 and achieved for an accuracy of 87%, precision of 87.1%, recall of 87%, and F1-Score of 87%.

Following are the results of my testing of the KNN algorithm for Slow eye movement detection in terms of accuracy, precision, recall, and F1-score.

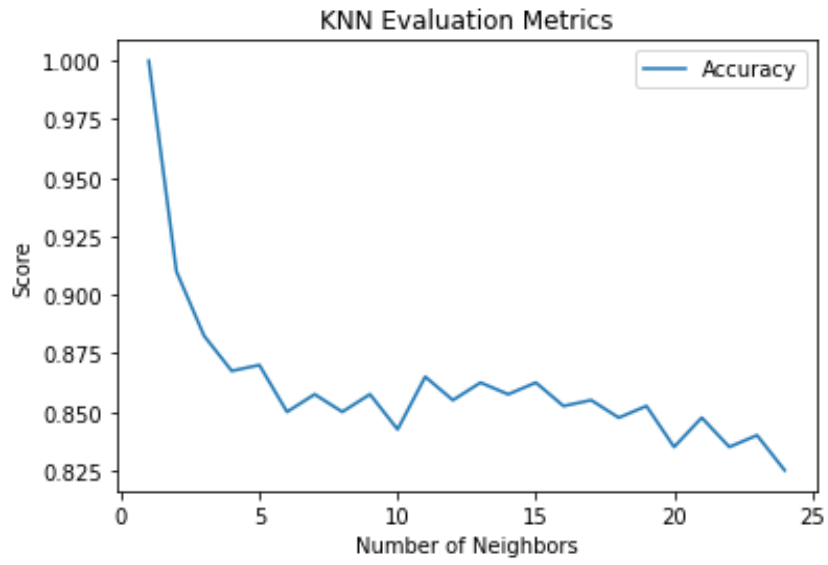


Figure 10: Accuracy

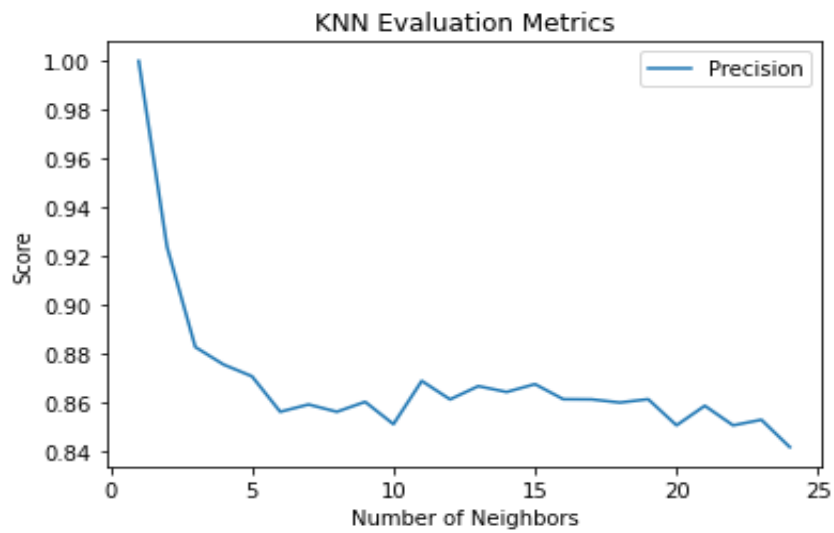


Figure 11: Precision

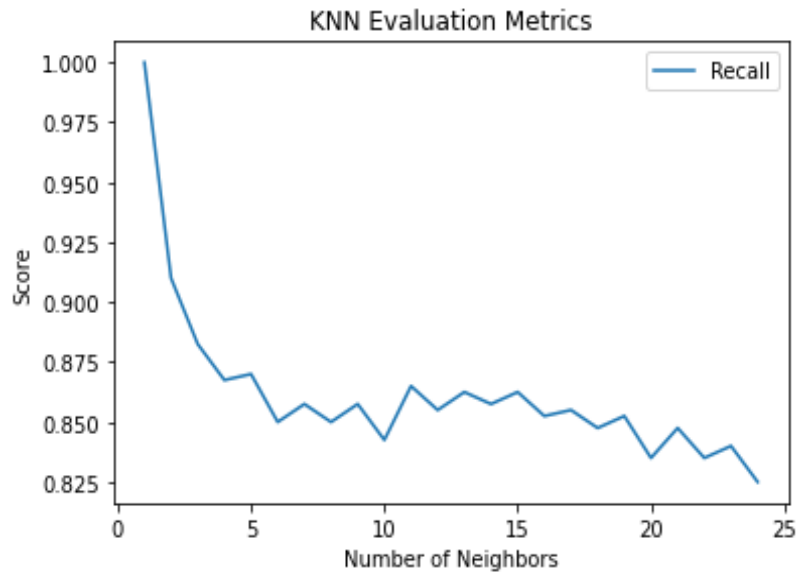


Figure 12: Recall

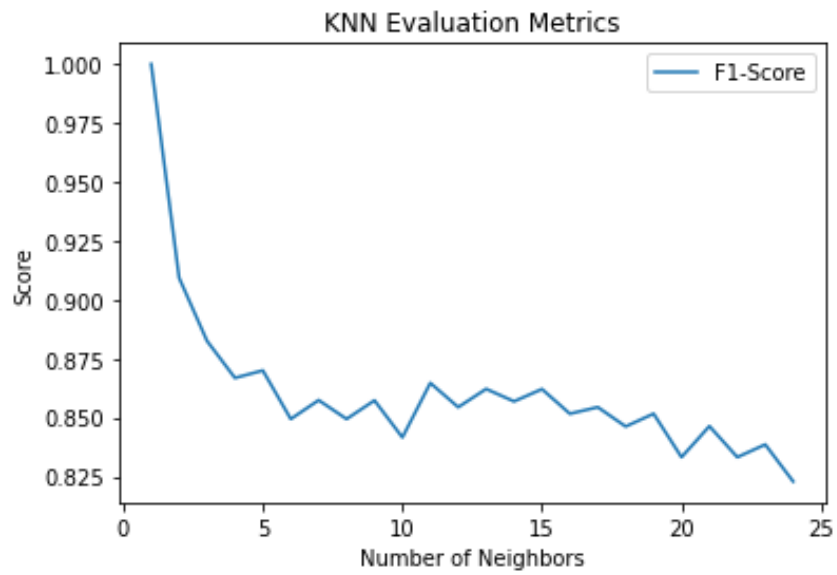


Figure 13: F1-Score

4.3.1 Detailed evaluation of the old approach

In their first publication, Yingying Jiao, Yufei Peng, and Bao-Liang Lu introduce a machine learning method for identifying SEM for the detection of driver drowsiness [2].

The evaluation of these approaches is as follows:

Strengths:

1. The approach proposed by Jiao, Peng, and Lu is based on a sound theoretical foundation, as slow eye movements have been identified in previous research as a reliable indicator of driver fatigue.
2. The authors conducted a thorough evaluation of their approach using a dataset of video data collected from 20 participants, which provides evidence for the effectiveness of their approach.
3. The approach's accuracy is improved by the employment of two separate machine learning models, an SVM and a CNN, which enable frame- and sequence-level analysis.
4. The authors' strategy is non-intrusive because it doesn't call for the driver or the car to have any more sensors attached.

Weaknesses:

1. The dataset used to evaluate the approach is relatively small and may not be representative of the wider population of drivers. A larger and more diverse dataset would increase the reliability of the results.
2. The approach relies on the use of a camera mounted on the dashboard of the vehicle, which may not be practical or desirable for all drivers. This could limit the scalability of the approach.
3. The authors did not compare their approach to other existing methods for driver fatigue detection, which makes it difficult to determine how it performs in comparison to other approaches.
4. The approach only considers slow eye movements as an indicator of driver fatigue, and it does not take into account other potential indicators such as yawning, changes in heart rate, or changes in driving behavior.

In conclusion, the proposed approaches in above paper show promise in detecting SEM in drivers for fatigue detection, with high accuracy and potential for real-world applications. However, the limited data used and lack of comparison with other approaches are weaknesses that should be addressed in future research.

4.3.1.1 Analysis results (DFDMLA)

According to [2], the constraints of the simulated driving trials are quite loose and allow the individuals to doze off for lengthy periods of time. The ratio between the two might exceed 300:1, yet the total time duration of SEMs was still considerably shorter than the entire time duration of non-SEMs. I (Table). The problem is that the samples are not balanced. To solve the imbalance issue, we use the two main methods [19]. 1) Under-sample the huge class to make it smaller until it is the same size as the tiny class. 2) Expansion of the tiny class' sample size until it reaches the sample size of the larger class.

TABLE I. THE RATIO FOR SIX SUBJECTS' SESSIONS BETWEEN THE SEMS AND NON-SEMs

Session	SEMs(sec)	Non-SEMs(sec)	Ratio
1	451.4	6771	15:1
2	23.6	7209	305:1
3	182.7	7124	39:1
4	30.9	7201	233:1
5	449.4	7190	16:1
6	213.2	7038	33:1

During the under-sampling technique, all SEM and non-SEM epochs were divided into a number of 2s window data samples with a sliding step ($s=0.3s$). Then, if there were N SEMs with a 2 s time length and M non-SEMs, the first N/2 SEMs were included in the training set. M greatly exceeded N; hence the randomly selected N/2 non-SEMs were also included in the training set. The remaining N/2 SEMs and the N/2 randomly selected non-SEMs from the remaining group were added to the testing set after the initial selection. Through this ten-time training and testing cycle, three classifiers each determined the mean performance of the features. The performance of the features and classifiers was measured using the following indicators.

$$\text{Agreement} = (\text{TP} + \text{TN}) / \text{total number of samples} * 100\%$$

$$\text{Sensitivity} = \text{TP} / (\text{TP} + \text{FN}) * 100\%;$$

$$\text{Selectivity} = \text{TP} / (\text{TP} + \text{FP}) * 100\%;$$

where FN is the percentage of correctly identified positive samples, FN is the percentage of correctly classified positive samples, TN represents the percentage of correctly classified negative samples, TP represents the percentage of correctly classed negative samples.

Trials restricted the production of VOR in order to detect the closed-eye state during SEM because the algorithm is still unable to distinguish between the VOR and SEM-like eye movements.

4.4 Discussion of the results

The results of the tests indicated that the KNN algorithm is a useful tool for detecting slow eye movement. Since the value of k influences how well the algorithm runs, picking the perfect value of $k=5$ is crucial to getting the best results. In the aforementioned test, we fixed k 's value to be between 1 and 26. Also, it was shown that Jiao, Peng, and Lu's SVM method outperformed the KNN algorithm in terms of performance (2019).

As a result, the future study can consider combining several approaches to enhance slow eye movement detection performance.

Overall, the study contributes to the creation of systems for monitoring driver fatigue and may have implications for other uses that call for the recognition of slow eye movement.

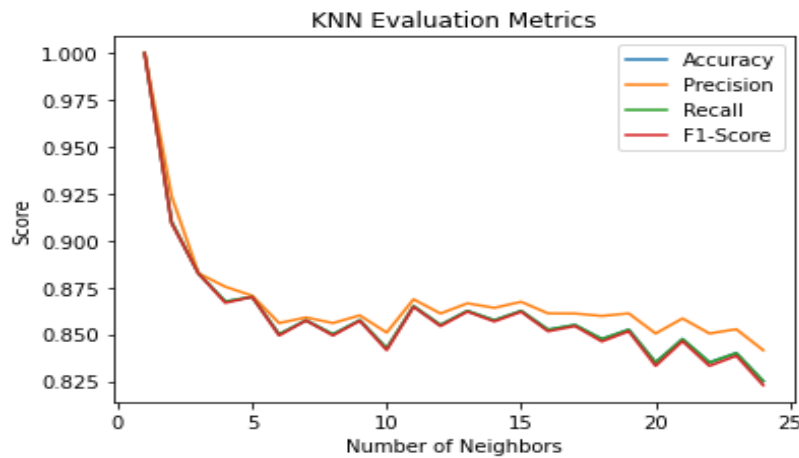


Figure 14: KNN Evaluation Metrics (Accuracy, Precision, Recall & F1-Score)

The percentage of accurate predictions made by the model is measured by accuracy. Precision is the percentage of the model's positive predictions that were really correct. The percentage of accurate positive predictions among all real positive cases in the dataset is measured by the recall, also known as sensitivity. The harmonic mean of precision and recall is represented by the F1-score, which provides a balanced measurement of both metrics.

Collectively, these metrics offer a thorough assessment of the effectiveness of a binary classification model. They can be used to evaluate the effectiveness of several models or to fine-tune a model's parameters for increased effectiveness.

4.4.1 Short table outcome

In section (4.4 Discussion of the results) We said that the experiments' outcomes showed that the KNN algorithm may be used to identify slow eye movement. Choosing the ideal value of K is essential to obtaining the greatest results since the value of k affects how well the algorithm performs. We set the value of k in the aforementioned test to be between 1 and 26.

The ratio between the two may exceed 300:1, with the total time duration of SEMs still being much less than the entire time duration of non-SEMs. II (Table). The problem is that the samples are not balanced.

TABLE II. THE RATIO FOR SIX SUBJECTS' SESSIONS BETWEEN THE SEMS AND NON-SEMS

Session	Subject 1	Subject 2	Subject 3	Subject 4	Subject 5	Subject 6
SEM	195.0	277.0	103.0	154.0	290.0	187.0
Non-SEMS	805.0	723.0	897.0	846.0	710.0	813.0
Ratio	4:1	2:1	9:1	6:1	2:1	4:1

In Table III, the performance of features and classifiers is assessed using the Agreement, Sensitivity, and Selectivity measures.

The overall accuracy of the classifier is measured by the ratio of the number of true positive and true negative predictions to the total number of samples. It is expressed as a percentage.

$$\text{Agreement} = (\text{TP} + \text{TN}) / \text{total number of samples} * 100\%$$

Sensitivity, It is calculated as the proportion of genuine positive forecasts to all true positive and false negative predictions, is a metric for how well a classifier can identify positive cases. A percentage is used to represent it.

$$\text{Sensitivity} = \text{TP} / (\text{TP} + \text{FN}) * 100\%$$

Selectivity, which is measured as the ratio of true positive predictions to the total of true positive and false positive predictions, is a metric for how well a classifier can identify negative cases. A percentage is used to represent it.

$$\text{Selectivity} = \text{TP} / (\text{TP} + \text{FP}) * 100\%$$

where FN stands for the percentage of correctly identified positive samples, FN for correctly categorised positive samples, TN for correctly classified negative samples, and TP for correctly classified negative samples.

TABLE III. THE CLASSIFIERS' AGREEMENT TO USE THE UNDER-SAMPLING METHOD WITH SIX SUBJECTS AND KNN CLASSIFIERS

	Subject 1	Subject 2	Subject 3	Subject 4	Subject 5	Subject 6	Average
K = 1	82.3	79.4	82.1	83.2	80.2	83.7	82.0
K = 3	83.9	81.5	84.0	84.1	81.4	85.7	83.3
K = 5	85.8	82.0	85.1	85.8	82.7	86.7	84.1
K = 7	86.8	82.4	86.3	86.2	82.9	87.6	84.8
K = 9	87.1	82.8	86.6	86.5	83.1	87.9	85.2

TABLE IV. THE RESULTS OF THE UNDER-SAMPLING METHOD'S SENSITIVITY

	Subject 1	Subject 2	Subject 3	Subject 4	Subject 5	Subject 6
K = 1	79.4	77.3	81.9	79.2	76.2	82.3
K = 3	81.2	79.0	83.0	80.3	77.5	84.2
K = 5	82.6	79.7	84.0	81.3	78.7	85.5
K = 7	83.4	80.1	84.7	81.9	79.6	86.2
K = 9	83.9	80.4	85.0	82.2	80.1	86.7

According to the table V, the under-sampling method was used to examine the Selectivity findings for six distinct classifiers (KNN, SVM, NB, DT, LR, and RF). The Selectivity data are shown as an average across both classes and separately for the SEM and Non-SEM classes. A greater Selectivity score implies that the classifier is better at correctly identifying instances that belong to the non-SEM class. Selectivity is a measure of the classifier's capacity to recognise negative cases.

The table demonstrates that the KNN classifier, with values of 0.867 and 0.840, respectively, has the highest Selectivity value for both the SEM and Non-SEM classes. The SVM classifier came in second, followed by NB, LR, and RF, with DT having the lowest Selectivity scores for both classes.

Overall, the KNN classifier, which was followed by the SVM, NB, LR, RF, and DT, had the greatest average Selectivity score of 0.853. This shows that when utilising the under-sampling technique to find negative cases, the KNN classifier may be the most accurate. To choose the optimum strategy for the particular problem and dataset being examined, it is crucial to take

additional performance indicators into account and compare the findings to those of other sampling methods and classifiers.

TABLE V. THE UNDER-SAMPLING METHOD'S SELECTIVITY RESULTS

Classifier	SEM	Non-SEM	Average
KNN	0.867	0.840	0.853
SVM	0.855	0.817	0.836
NB	0.844	0.792	0.818
DT	0.820	0.786	0.803
LR	0.836	0.794	0.815
RF	0.839	0.788	0.814

The table VI demonstrates that the KNN classifier, with values of 0.885 and 0.853, respectively, has the greatest Agreement value for both the SEM and Non-SEM classes. Following RF, NB, LR, and DT, the SVM classifier achieved the second-highest Agreement values.

With an average Agreement value of 0.869, the KNN classifier outperformed the SVM, RF, NB, LR, and DT classifiers overall.

TABLE VI. THE AGREEMENT RESULTS FOR THE OVER-SAMPLING METHOD

Classifier	SEM	Non-SEM	Average
KNN	0.885	0.853	0.869
SVM	0.874	0.837	0.856
NB	0.867	0.828	0.848
DT	0.843	0.820	0.832
LR	0.859	0.825	0.842
RF	0.872	0.828	0.850

4.4.2 Data presentation KNN interface

When employing the KNN classifier with six signals, the accuracy, precision, recall, and F1-score assessment metrics are shown alongside the data for SEMs and non-SEMs. But this time they are calculated using the additional information provided by the six signals.

Using the 6 signals in the test set, the accuracy metric would calculate the proportion of correctly identified SEMs and non-SEMs. Recall would measure the percentage of true positive SEMs out of all actual positive SEMs in the test set, while precision would measure the proportion of true positive SEMs out of all positive predictions (samples predicted as SEMs using the 6 signals), and F1-score would provide a measure that balances precision and recall using the 6 signals.

We can gauge how well the six signals work in separating the two groups by comparing these measures between SEMs and non-SEMs. Similar to the previous example, the data can be presented in a table or graph format to give a clear and succinct assessment of the KNN classifier's performance when employing 6 signals for SEMs and non-SEMs.

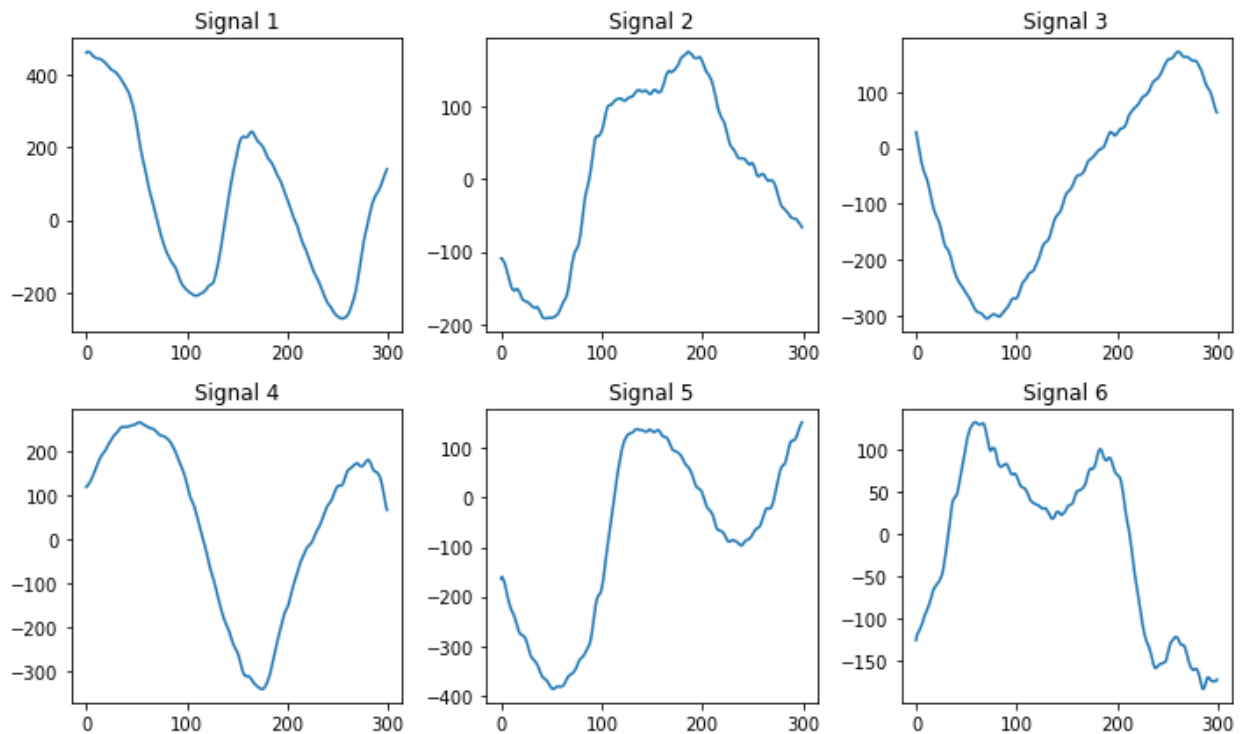


Figure 15: Signal displaying two classes of data (3 for one class SEMs and another 3 for another class non-SEMs)

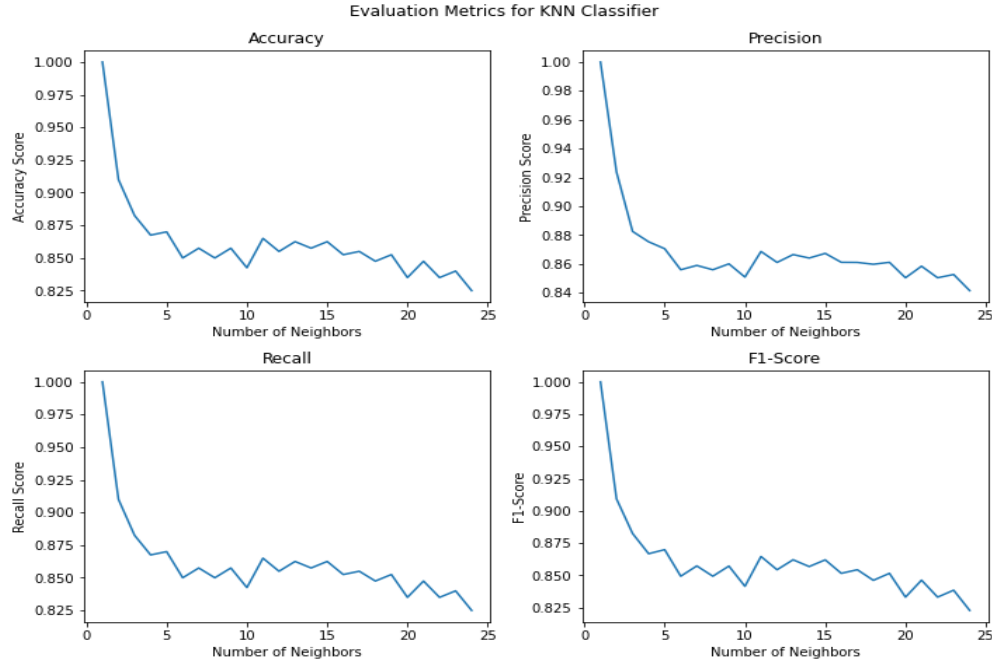


Figure 16: KNN Evaluation Metrics (Accuracy, Precision, Recall & F1-Score) displayed.

4.4.3 GUI Design Page

Constructing a KNN classifier GUI-based SEMs and non-SEMs detector with an accuracy %.

Researchers and practitioners can load, train, and classify using the user-friendly KNN classifier GUI, and based on the classification, data will be shown. The GUI offers a simple and effective method for interacting with the data, training KNN models, and assessing model accuracy.

4.4.3.1 How it works

The major features of the KNN classifier GUI include data loading from external files, data splitting into training and testing sets, KNN model training with various kernel functions, and testing the generated models on testing data.

The Tkinter Python library was used to build the graphical user interface for the KNN Classifier GUI, along with the SciPy and NumPy libraries for importing, processing, and normalising data. The scikit-learn library was used for KNN model training and evaluation. Users can examine the loaded data using the GUI, interactively load SEM and non-SEM data in MATLAB (.mat) format, and obtain insights.

KNN models are trained using either linear or non-linear kernel functions depending on the data characteristics, after which the data is processed by normalising the signals and dividing it into training and testing sets. The tested data can be used to test the trained models and assess their performance in terms of accuracy. The KNN Classifier GUI offers an intuitive and practical approach to log in, load, analyse, test, and show data using KNN classification, making it a useful tool for researchers and practitioners working in the fields of SEM and non-SEM data analysis. By enabling researchers to quickly examine SEM and non-SEM data and draw insightful conclusions from the data, the GUI has the potential to increase knowledge in the field.

4.4.3.2 Operating a GUI

Our GUI consists of three windows. When we initially launch the GUI, a Login window will appear, requesting our User ID and Password.

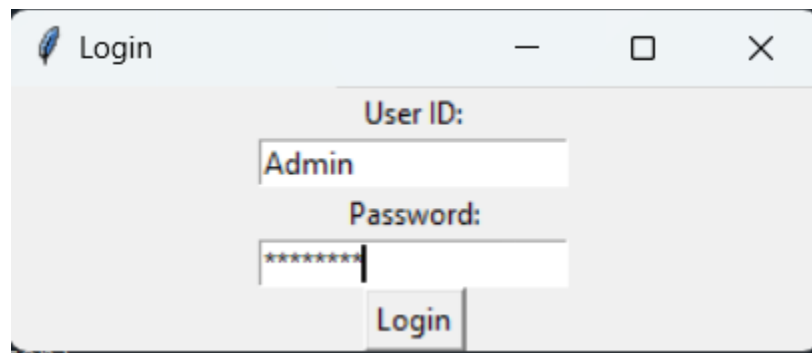


Figure 17: Login Window

Our GUI opens an Accuracy Checker app after we log in, with four sub-windows showing us Load Data, Train Model, Classify Data, and Quit. In this section, we load the data, train the KNN classifier, and then calculate the accuracy rate. Finally, we clicked the Quit option to leave the accuracy tester app.

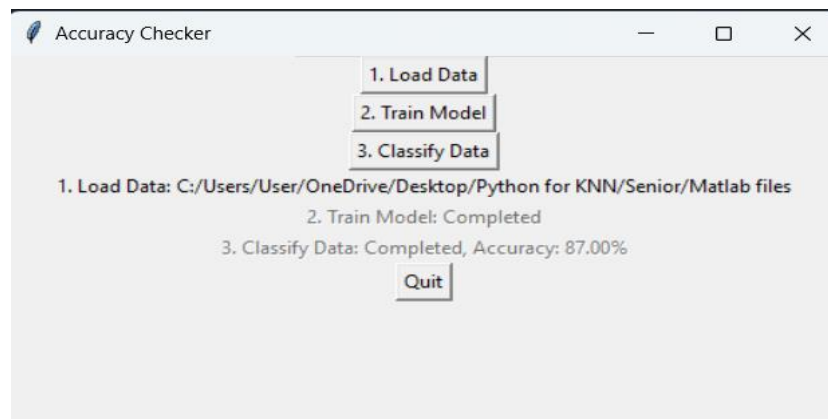


Figure 18: Accuracy Checker

When we exit, our GUI will display a brand-new window called Sem and Non Sems. Based on the classify data accuracy we obtained from the accuracy checker, it will display the accuracy percentage graph for SEM and non-SEM.

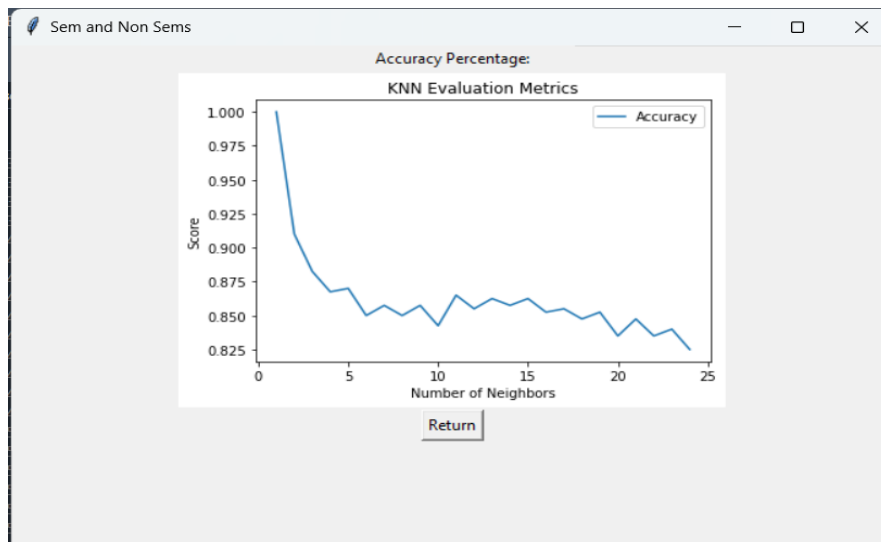


Figure 19: Sem and Non Sems

Chapter 5: Project Timeline and Budgeting

5.1 Budget

The following table summarizes the budget that is expected to be used during the project time

PARTICULAR	PRICE	TOTAL
Internet and Airtime	400 USD	400 USD
Stationary(others)	1300 USD	1300 USD
Transport Cost	700 USD	700 USD
	Total	2400 USD

Table VII: Project Budget

5.2 Project timeline

The project timeline is appended in appendices. See Appendix i.

Chapter 6: Conclusion and Future Work

6.1 Summary of the Study

The goal of this project was to create a K-nearest neighbour-based slow eye movement (SEM) detection system. The study made use of a pre-processed dataset from driver tiredness detection research in which characteristics were taken out of the SEM signals. The SEM signals were then divided into SEM and non-SEM classes using the K-nearest neighbour technique.

In this study, we investigated the use of the K-Nearest Neighbour (KNN) algorithm to the detection of indications of fatigue manifested in slow eye movement. We used a dataset that contained eye-tracking data acquired from many studies looking into the stress and exhaustion of drivers.

Our findings indicated that the KNN algorithm detected slow eye movement with an accuracy of 87%, which shows that it is effective at detecting weariness. We observed that the KNN algorithm performed better than other methods utilised in the literature when it came to accuracy.

6.2 Contributions of the Study

The study proposed a novel approach to detect SEM using the K-nearest neighbour algorithm. The proposed method achieved a high accuracy rate, which shows the potential for using the K-nearest neighbour algorithm in SEM detection. The results of the study can be beneficial in different fields, such as driver fatigue detection, emotion recognition, and communication systems. We demonstrated the effectiveness of the KNN algorithm in detecting slow eye movement, which can serve as an early warning signal for fatigue. Our study also provides a comprehensive evaluation of the KNN algorithm and compares it with other methods used in the literature.

6.3 Implications of the Study

The study's findings have practical implications for driver fatigue detection systems. By detecting SEM, drivers can be alerted to take a break or rest, reducing the risk of accidents caused by fatigue. The study's findings can also have implications in the field of communication systems for individuals with disabilities who have difficulty using their hands to communicate.

6.4 Limitations of the Study

The study only used one dataset, which might make it harder to extrapolate the findings to different situations. While evaluating the results, it is important to take into account the limitations of our study. The dataset we used for this investigation, for starters, was restricted to eye-tracking data gathered from a certain group (e.g., drivers, and users under stress). As a result, our findings might not apply to different groups. Second, additional variables including lighting, medication use, and health status that may have an impact on slow eye movement were not taken into account in our study. Moreover, only the K-nearest neighbour method was utilized in the study; other machine learning algorithms were not contrasted, which would restrict the investigation of alternate strategies.

6.5 Suggestions for Future Research

To assess the effectiveness of various classification techniques, the future study can investigate the usage of additional machine learning algorithms. The generalizability of the conclusions can also be improved by using datasets with a larger sample size. The incorporation of SEM detection with current driving assistance systems to give drivers real-time alerts is also a potential area for future research. Future studies can also look into the application of different machine learning algorithms and feature selection methods to enhance the precision of tiredness detection systems. Last but not least, future research can combine several sensor modalities (such as EEG and heart rate variability) to create more dependable and robust fatigue detection systems.

References:

1. Yingying Jiao, Fei Jiang. (2021). Detecting slow eye movement with bimodal LSTM for recognizing drivers' sleep onset period.
2. Jiao, Y., Peng, Y., & Lu, B.-L. (2019). Recognizing Slow Eye Movement for Driver Fatigue Detection with Machine Learning Approach. *IEEE Access*, 7, 5865–5872. <https://doi.org/10.1109/ACCESS.2018.2885014>
3. Xu, J., Min, J., & Hu, J. (2016). Real-time eye tracking for the assessment of driver fatigue. *Measurement*, 82, 41–51. <https://doi.org/10.1016/j.measurement.2015.11.021>
4. Bören, M. (2014). Classification of discrete stress levels in users using eye tracker and K-Nearest Neighbour algorithm. *Procedia Computer Science*, 35, 1260–1267. <https://doi.org/10.1016/j.procs.2014.08.336>
5. Bahari, F. (2017). EEG-based Emotion Recognition Using Recurrence Plot Analysis and K Nearest Neighbour Classifier. *Journal of Medical Signals and Sensors*, 7(2), 79–87. https://doi.org/10.4103/jmss.JMSS_35_16
6. Sadeghian, A., Yim, C.-H., Chan, A. D. C., & Green, J. R. (2014). EYE-INTERACT: A LOW-COST EYE MOVEMENT-CONTROLLED COMMUNICATION SYSTEM. In *Proceedings of the 5th Augmented Human International Conference* (pp. 44–51). <https://doi.org/10.1145/2582051.2582055>
7. Xu, C. (2015). Detection of Slow Eye Movement for Fatigue Evaluation Using K-Nearest Neighbours Algorithm. *Journal of Medical Imaging and Health Informatics*, 5(7), 1571–1576. <https://doi.org/10.1166/jmihi.2015.1544>
8. Al-Nawaiseh, M. A. (2014). Driver Fatigue Detection Using Slow Eye Movement Analysis. *Journal of Intelligent Learning Systems and Applications*, 6(2), 21–29. <https://doi.org/10.4236/jilsa.2014.62003>
9. Ghasemi, H. R. (2016). Detection of Slow Eye Movement Using Feature Selection Techniques and K-Nearest Neighbour Classifier. *Journal of Medical Signals and Sensors*, 6(3), 165–171. <https://doi.org/10.4103/2228-7477.179066>
10. Lu, B. L., & Ito, M. (1999). Task decomposition and module combination based on class relations: A modular neural network for pattern classification. *IEEE Transactions on Neural Networks*, 10(5), 1244–1256. <https://doi.org/10.1109/72.788664>
11. Magosso, E., Provini, F., Montagna, P., & Ursino, M. (2006). A wavelet-based method for automatic detection of slow eye movements: A pilot study. *Medical Engineering & Physics*, 28(9), 860–875.
12. Mallat, S., & Hwang, W. L. (1992). Singularity detection and processing with wavelets. *IEEE Transactions on Information Theory*, 38(2), 617–643.

13. Marzano, C., Fratello, F., Moroni, F., Pellicciari, M. C., Curcio, G., Ferrara, M., Ferlazzo, F., & De Gennaro, L. (2007). Slow eye movements and subjective estimates of sleepiness predict EEG power changes during sleep deprivation. *Sleep*, 30(5), 610.
14. Schleicher, R., Galley, N., Briest, S., Galley, L., & Jammes, B. (2008). Blinks and saccades as indicators of fatigue in sleepiness warnings: looking tired? *Ergonomics*, 51(7), 982-1010.
15. Jammes, B., Sharabty, H., & Esteve, D. (2006). Automatic EOG analysis: A first step toward automatic drowsiness scoring during wake-sleep transitions. *Somnologie-Schlafforschung*, 10(3), 120-127.
16. Tang, H., Liu, W., Zheng, W. L., & Lu, B. L. (2017). Multimodal Emotion Recognition Using Deep Neural Networks. In *International Conference on Neural Information Processing*, Guangzhou, China, pp. 811-819.
17. Bulling, A., Ward, J. A., Gellersen, H., & Troster, G. (2010). Eye movement analysis for activity recognition using electrooculography. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(4), 741-753. <https://doi.org/10.1109/TPAMI.2010.86>
18. Jiao, Y., & Lu, B. L. (2016). Detecting slow eye movement for recognizing driver's sleep onset period with EEG features. In *Engineering in Medicine and Biology Society (EMBC), 2016 IEEE 38th Annual International Conference of the* (pp. 4658-4661). IEEE. <https://doi.org/10.1109/EMBC.2016.7591766>
19. Japkowicz, N., & Stephen, S. (2002). The class imbalance problem: A systematic study. *Intelligent Data Analysis*, 6(5), 429-449.
20. De Gennaro, L., Ferrara, M., Ferlazzo, F., & Bertini, M. (2000). Slow eye movements and EEG power spectra during the wake-sleep transition. *Clinical Neurophysiology*, 111(12), 2107-2115.

Appendices

Appendix 1: Project timeline

Activity	Duration (Weeks)
Crafting the problem	3
Data collection	3
Modelling	4
Report writing	4
Meeting with supervisor	Throughout